Favoritos na tela Detalhe do Produto

Etapa 1 — Ajustar os imports

No início de detalhe-produto.tsx, substitua apenas os imports correspondentes e mantenha os demais:

```
import Router, { useState, useEffect, useRef } from "react";
import { router, useLocalSearchParams, useFocusEffect } from "expo-router";
import AsyncStorage from "@react-native-async-storage/async-storage";
```

Etapa 2 — Acrescentar os estados padronizados

Dentro de DetalheProdutoScreen(), abaixo dos estados existentes, adicione:

```
const [mensagemFavorito, setMensagemFavorito] = useState("");
const [processandoFavoritos, setProcessandoFavoritos] = useState<number[]>([]);
const bloqueiosFavoritos = useRef<Set<number>>(new Set());
```

Etapa 3 — Manter o carregamento do produto e preparar a estrela

No useEffect já existente, mantenha a consulta de produto e substitua apenas o setProduto(json.data) por:

```
setProduto({ ...json.data, favorito: false });
```

Isso inicializa a estrela vazia enquanto o status real ainda não foi consultado.

Etapa 4 — Criar carregarFavoritos()

Abaixo do useEffects(), antes dos retornos condicionais if (carregando) e if (erro || !produto), acrescente a função abaixo:

```
async function carregarFavoritos(idProduto: number) {
  try {
    const token = await AsyncStorage.getItem("token");
    if (!token) {
      router.replace("/");
      return;
    }

    const resposta = await fetch(`${API}/favoritos`, {
      headers: {
        Accept: "application/json",
        Authorization: `Bearer ${token}`,
      },
    });
    const json = await resposta.json();
    if (!resposta.ok || !json.success) {
      setMensagemFavorito(json.message || "Erro ao carregar favoritos.");
      return;
    }

    const idsFavoritos = new Set(
      (json.data ?? []).map((item: any) => Number(item.id_produto))
    );
    setProduto((anterior: any) =>
      anterior && Number(anterior.id_produto) === idProduto
        ? { ...anterior, favorito: idsFavoritos.has(idProduto) }
```

```
        : anterior
    );
  } catch (erro) {
    console.log("Erro ao carregar favoritos:", erro);
    setMensagemFavorito("Não foi possível consultar os favoritos.");
  }
}
```

Etapa 5 — Consultar favoritos ao abrir ou retornar ao detalhe

Adicione após a função anterior este useFocusEffect antes dos retornos condicionais. Ele é executado quando a tela recebe foco e quando muda o ID do produto:

```
useFocusEffect(
  React.useCallback(() => {
    if (produto?.id_produto) {
      carregarFavoritos(Number(produto.id_produto));
    }
  }, [produto?.id_produto])
);
```

ATENÇÃO: como o exemplo usa React.useCallback, caso ainda não tenha, acrescente React ao primeiro import, ficando:

```
import React, { useState, useEffect, useRef } from "react";
```

Etapa 6 — Criar alterarFavorito() com POST e DELETE

Ainda antes dos retornos condicionais, adicione a função completa:

```
async function alterarFavorito(idProduto: number) {
  if (!produto || bloqueiosFavoritos.current.has(idProduto)) return;
  bloqueiosFavoritos.current.add(idProduto);
  setProcessandoFavoritos((atual) => [...atual, idProduto]);
  setMensagemFavorito("");

  try {
    const token = await AsyncStorage.getItem("token");
    if (!token) {
      router.replace("/");
      return;
    }

    const jaFavorito = Boolean(produto.favorito);
    const resposta = await fetch(
      jaFavorito ? `${API}/favoritos/${idProduto}` : `${API}/favoritos`,
      {
        method: jaFavorito ? "DELETE" : "POST",
        headers: {
          Accept: "application/json",
          Authorization: `Bearer ${token}`,
          ...(!jaFavorito ? { "Content-Type": "application/json" } : {}),
        },
        ...(!jaFavorito
          ? { body: JSON.stringify({ id_produto: idProduto }) }
          : {}),
      }
    );

    const json = await resposta.json();
    if (!resposta.ok || !json.success) {
```

```
      setMensagemFavorito(json.message || "Erro ao atualizar favorito.");
      return;
    }
    setProduto((anterior: any) =>
      anterior && Number(anterior.id_produto) === idProduto
        ? { ...anterior, favorito: !jaFavorito }
        : anterior
    );
  } catch (erro) {
    console.log("Erro ao alterar favorito:", erro);
    setMensagemFavorito("Não foi possível conectar ao servidor.");
  } finally {
    bloqueiosFavoritos.current.delete(idProduto);
    setProcessandoFavoritos((atual) =>
      atual.filter((id) => id !== idProduto)
    );
  }
}
```

Se ainda não é favorito, envia POST com id_produto. Se já é, envia DELETE. O bloqueio é removido em finally, mesmo em caso de erro.

Etapa 7 — Substituir apenas o Pressable da estrela

No cabeçalho do produto, substitua a estrela fixa por:

```
<Pressable
  style={detalheProdutoStyle.btnFavorito}
  onPress={() => alterarFavorito(Number(produto.id_produto))}
  disabled={processandoFavoritos.includes(Number(produto.id_produto))}
>
  <Text style={detalheProdutoStyle.iconeFavorito}>
    {produto.favorito ? "★" : "☆"}
  </Text>
</Pressable>
```

O layout e os estilos existentes são preservados. A estrela muda somente após resposta de sucesso.

Etapa 8 — Exibir mensagens de erro

Logo após o fechamento do cabeçalho, antes de <View style={detalheProdutoStyle.main}>, adicione:

```
{mensagemFavorito ? (
  <Text style={detalheProdutoStyle.erroFavorito}>
    {mensagemFavorito}
  </Text>
) : null}
```

Em src/styles/detalheProdutoStyle.ts, dentro do StyleSheet.create, acrescente:

```
erroFavorito: {
  color: "#CC0000",
  fontSize: 14,
  textAlign: "center",
  marginVertical: 10,
},
```